KANT AX/AI 엔지니어 트랙 · 30

Docker 2

내 프로그램을 이미지로 만들기

exec, 볼륨, Dockerfile, compose

```
~/shopping-cart $ docker build -t cart .
[+] Building 5.6s (9/9) FINISHED

~/shopping-cart $ docker run cart
텀블러 2개를 담았습니다.
무선 이어폰 1개를 담았습니다.
품절된 상품입니다.
----------------------------------------
텀블러          x 2    =        30,000원
무선 이어폰      x 1    =        89,000원
```

오늘의 순서

1	**컨테이너 안에 들어가기** `docker exec -it`, 지우면 사라지는 변경	6–9쪽
2	**볼륨으로 남기기** `-v 볼륨이름`, `-v ./폴더`, `docker volume ls`	10–14쪽
3	**Dockerfile로 이미지 만들기** `FROM` `WORKDIR` `COPY` `RUN` `CMD`, `build`, `--rm`, `.dockerignore`	15–24쪽
4	**docker compose 맛보기** `compose.yaml`, `up -d`, `ps`, `down`	25–26쪽
5	**자주 나는 오류** `build`, `run`, `exec`, `-v` 오류	27–29쪽
6	**실제 상황 예제** 메뉴판 고치기, 친구에게 건네기, 정리하기	30–32쪽

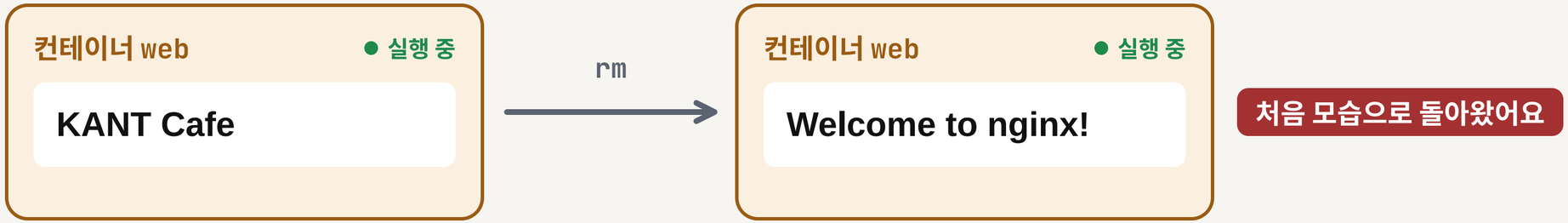
3–5쪽은 들어가기, 33–34쪽은 막힐 때와 한 장 요약이에요.

들어가기

29가 끝날 때 남은 질문이 두 개예요

1 컨테이너를 지우면 안에서 바꾼 것도 사라져요

지워도 남게 하려면 어떻게 할까요?



---

2 29의 이미지는 모두 남이 만든 것이었어요

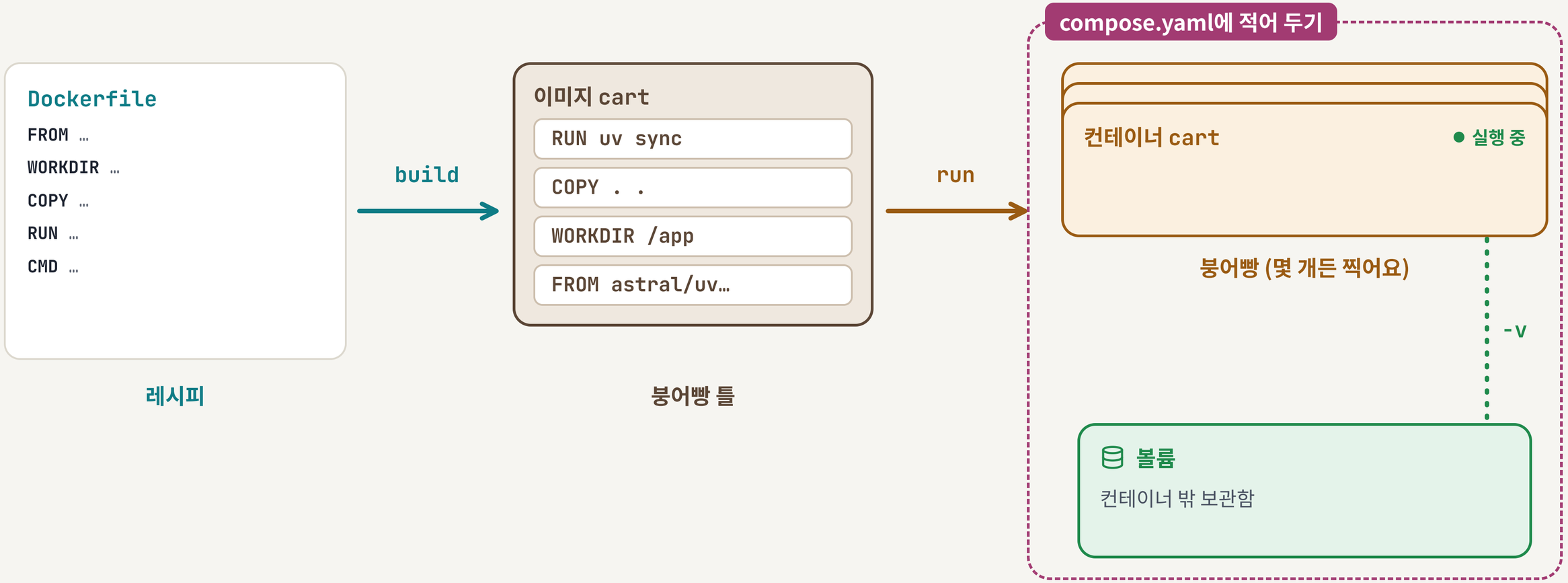
28의 장바구니도 이미지로 만들 수 있을까요?

`nginx` `hello-world` `python:3.12-slim` 다음은 `cart`

두 질문의 답이 볼륨과 Dockerfile이에요.

들어가기

레시피로 틀을 굽고, 틀로 붕어빵을 찍어요



1파트 exec는 붕어빵 안, 2파트 볼륨은 붕어빵 밖, 3파트 Dockerfile은 틀, 4파트 compose는 찍는 방법이에요.

들어가기

실습 폴더 두 개를 준비해요

```
~ $ mkdir cafe
~ $ cd cafe
~/cafe $ mkdir site
```

1, 2, 4파트

```
~ $ cd shopping-cart
```

3파트: 28의 장바구니

```
~/shopping-cart $ ls
__pycache__  cart.py  main.py  product.py  pyproject.toml  uv.lock
```

```
~/
  cafe/
    site/
  shopping-cart/
    main.py cart.py product.py
    pyproject.toml uv.lock
```

Windows Git Bash는 새 창을 열 때마다 이 줄을 먼저 쳐요.

```
~ $ export MSYS_NO_PATHCONV=1
```

Git Bash가 /usr/share 같은 컨테이너 경로를
Windows 경로로 바꿔 버리는 것을 막아요.

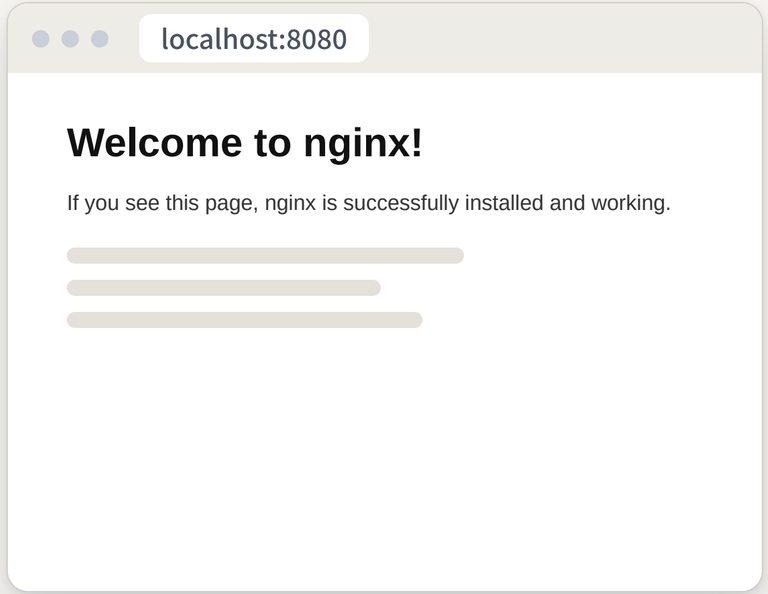
1. 컨테이너 안에 들어가기

29처럼 nginx 컨테이너를 하나 띄워요

```
~/cafe $ docker run -d --name web -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
6d2e43921e9e: Pull complete
37fb86c2a767: Pull complete
1962e1ff75b4: Pull complete
1724c05f72c6: Pull complete
bbb080c71d50: Pull complete
512ac5e7ce8e: Pull complete
7cb4701ce7cd: Pull complete
d2c7e31f04a3: Download complete
4582dace95e2: Download complete
Digest: sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
Status: Downloaded newer image for nginx:latest
b0ed4b95660eb4401f07c7486ea8f1782f84812935ec4e68a1d4af3d622ce8ff
```

29 끝에서 지워서 다시 받아요

컨테이너 ID



1. 컨테이너 안에 들어가기

docker exec -it로 실행 중인 컨테이너 안에 들어가요

```
docker exec -it web bash
```

- `exec` — 실행 중인 컨테이너에서
- `-it` — 키보드로 주고받기
- `web` — 컨테이너 이름
- `bash` — 열 프로그램 (셸)

29에서는

`docker run -it ... bash`

새 컨테이너를 만들면서 들어갔어요.
exec는 켜 둔 컨테이너에 들어가요.

```
~/cafe $ docker exec -it web bash
root@b0ed4b95660e:/#
```

@ 뒤는 앞 장의 컨테이너 ID 앞 12글자예요.

컨테이너 web — ● 실행 중

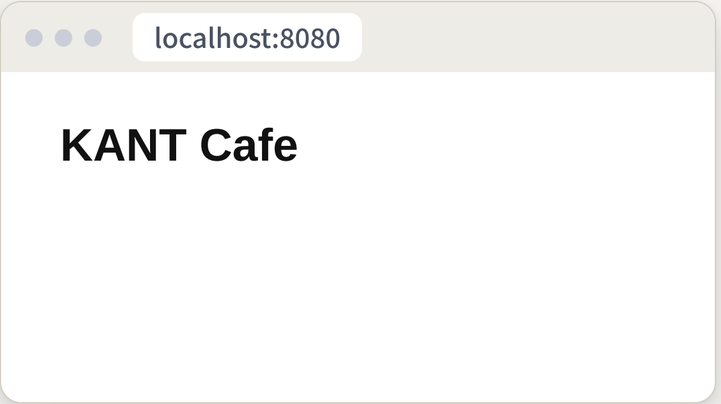
`root@…:/#`

지금 치는 명령은
컨테이너 안의 리눅스에서 돌아요

1. 컨테이너 안에 들어가기

안에서는 01에서 배운 명령이 그대로 통해요

```
root@b0ed4b95660e:/# ls                                          ← 컨테이너 안의 맨 위 폴더(/)
bin   docker-entrypoint.d   home   media  proc  sbin  tmp
boot  docker-entrypoint.sh  lib    mnt    root  srv   usr
dev   etc                   lib64  opt    run   sys   var
root@b0ed4b95660e:/# cd /usr/share/nginx/html                    ← nginx가 보여 주는 폴더
root@b0ed4b95660e:/usr/share/nginx/html# ls
50x.html  index.html
root@b0ed4b95660e:/usr/share/nginx/html# echo "<h1>KANT Cafe</h1>" > index.html
root@b0ed4b95660e:/usr/share/nginx/html# cat index.html
<h1>KANT Cafe</h1>                                               ← index.html 내용 바꾸기
root@b0ed4b95660e:/usr/share/nginx/html# exit                    ← 내 컴퓨터로 돌아오기
exit
```

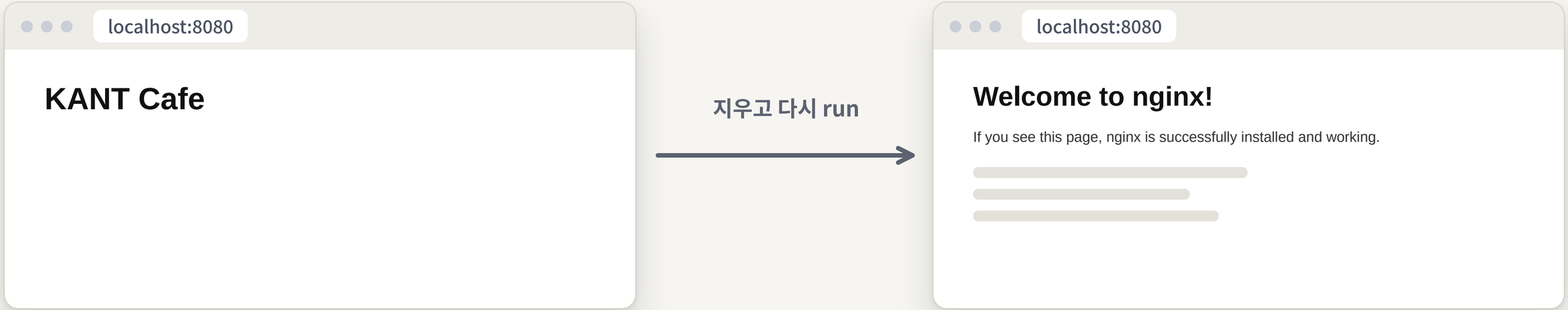


새로 고치면 바뀐 페이지가 보여요.

컨테이너를 지우면 안에서 바꾼 것도 같이 사라져요

```
~/cafe $ docker stop web
web
~/cafe $ docker rm web
web
~/cafe $ docker run -d --name web -p 8080:80 nginx
9a60b3514b1b48dcc5e687ef10df94b5ea449cc9c62e5fc723a5f912c234a7e2
```

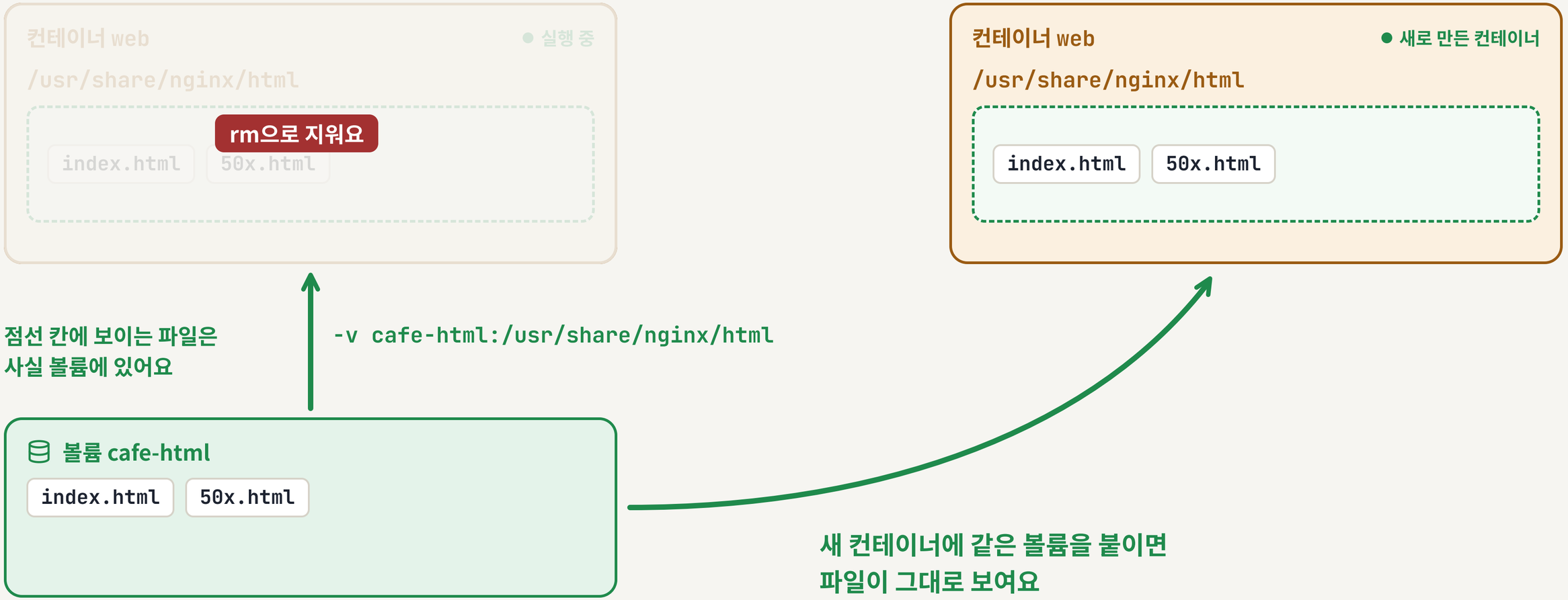
같은 이름, 새 컨테이너



안에서 바꾼 파일은 그 붕어빵에만 있었어요.

2. 볼륨으로 남기기

볼륨은 컨테이너 밖에 두는 보관함이에요



2. 볼륨으로 남기기

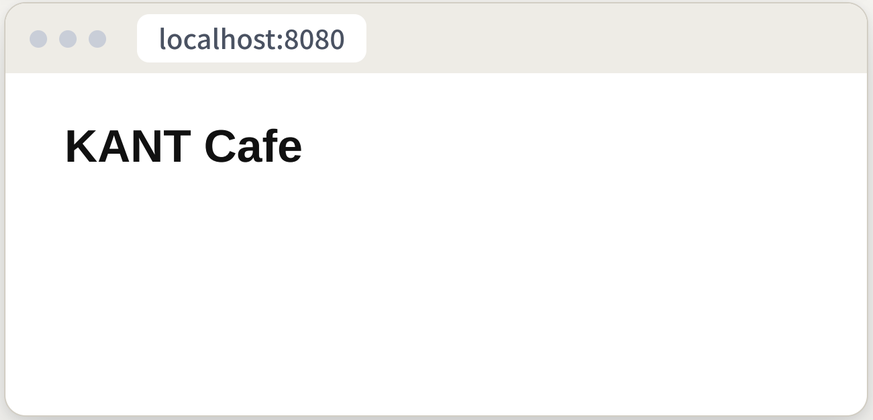
-v 볼륨이름:경로를 붙이면 지웠다 만들어도 남아요

```
~/cafe $ docker run -d --name web -p 8080:80 -v cafe-html:/usr/share/nginx/html nginx
fb5b80f6c2df…
~/cafe $ docker exec -it web bash
root@fb5b80f6c2df:/usr/share/nginx/html# echo "<h1>KANT Cafe</h1>" > index.html
root@fb5b80f6c2df:/usr/share/nginx/html# exit
exit
~/cafe $ docker stop web; docker rm web
web
web
~/cafe $ docker run -d --name web -p 8080:80 -v cafe-html:/usr/share/nginx/html nginx
de9fdf293a7a…
```

1파트처럼 index.html 바꾸기

지우고

지웠다 다시 만들어도
KANT Cafe가 그대로예요.



2. 볼륨으로 남기기

docker volume ls로 남아 있는 볼륨을 봐요

```
~/cafe $ docker volume ls
DRIVER    VOLUME NAME
local     cafe-html
```

-v에 적은 이름

Docker Desktop 왼쪽 **Volumes** 메뉴에서도 보여요.

`docker volume ls`	볼륨 목록 보기
`docker volume rm 이름`	볼륨 지우기 (붙은 컨테이너를 먼저 지워요)

2. 볼륨으로 남기기

-v ./폴더:경로는 내 폴더를 컨테이너에 그대로 연결해요

```
~/cafe $ docker run -d --name web -p 8080:80 -v ./site:/usr/share/nginx/html nginx
```

site/index.html

```
1  <meta charset="utf-8">
2  <h1>KANT 카페 메뉴</h1>
3  <p>아메리카노 4,500원</p>
4  <p>카페라테 5,000원</p>
```

./로 시작하면 내 폴더, 아니면 볼륨 이름이에요.

localhost:8080

KANT 카페 메뉴

아메리카노 4,500원

카페라테 5,000원

VS Code에서 저장하고 새로 고치기만 하면 돼요.

2. 볼륨으로 남기기

남길 때는 볼륨, 고치면서 볼 때는 폴더 연결을 써요

	볼륨 이름	내 폴더 연결
쓰는 모양	`-v cafe-html:경로`	`-v ./site:경로`
파일이 있는 곳	Docker가 관리하는 곳	내 폴더 (VS Code로 열려요)
목록 보기	`docker volume ls`	`ls site`
고르는 때	컨테이너가 만든 데이터를 남길 때	내가 고친 파일을 바로 보고 싶을 때

4파트 compose와 6파트 예제에서는 폴더 연결을 써요.

3. Dockerfile로 이미지 만들기

장바구니를 친구 컴퓨터에서 돌리려면 준비할 게 많아요

지금까지 (09, 28)

1. 파이썬 3.14 설치
2. uv 설치
3. `uv sync`
4. `uv run main.py`

Windows와 Mac, 파이썬 버전마다
막히는 곳이 달라요.

이미지로 건네면

이미지 `cart`

- uv sync 끝난 상태
- 장바구니 코드
- 파이썬 3.14 + uv

1. `docker run cart`

Docker만 있으면 어느 컴퓨터에서나 같은 결과예요.

3. Dockerfile로 이미지 만들기

Dockerfile은 다섯 줄이에요

Dockerfile

```
1 FROM astral/uv:python3.14-bookworm-slim
2 WORKDIR /app
3 COPY . .
4 RUN uv sync
5 CMD ["uv", "run", "main.py"]
```

어떤 틀 위에서 시작할까 파이썬과 uv가 든 이미지

안에서 일할 폴더 없으면 만들고 cd 해요

내 파일을 넣기 지금 폴더를 /app으로

만들 때 한 번 실행 패키지 맞추기

찍을 때마다 실행 켜지면 칠 명령

파일 이름은 확장자 없이 `Dockerfile`, D는 대문자예요.

shopping-cart 폴더 맨 위, main.py 옆에 VS Code로 새 파일을 만들어요.

3. Dockerfile로 이미지 만들기

FROM: uv를 만든 회사의 이미지 위에서 시작해요

```
FROM astral/uv:python3.14-bookworm-slim
```

- `astral/uv` — 만든 곳/이미지 이름
- `python3.14` — 파이썬 버전
- `bookworm-slim` — 리눅스 종류 (가벼운 판)

29에서 본 이미지 이름 규칙 그대로예요.

이름 : 태그

Docker Hub

astral/uv

The official Docker image for uv, the Python package manager.

인증 마크는 없어요. uv를 만든 회사 Astral이 올렸고 uv 공식 문서가 이 이미지를 안내해요.

`.python-version`

```
3.14
```

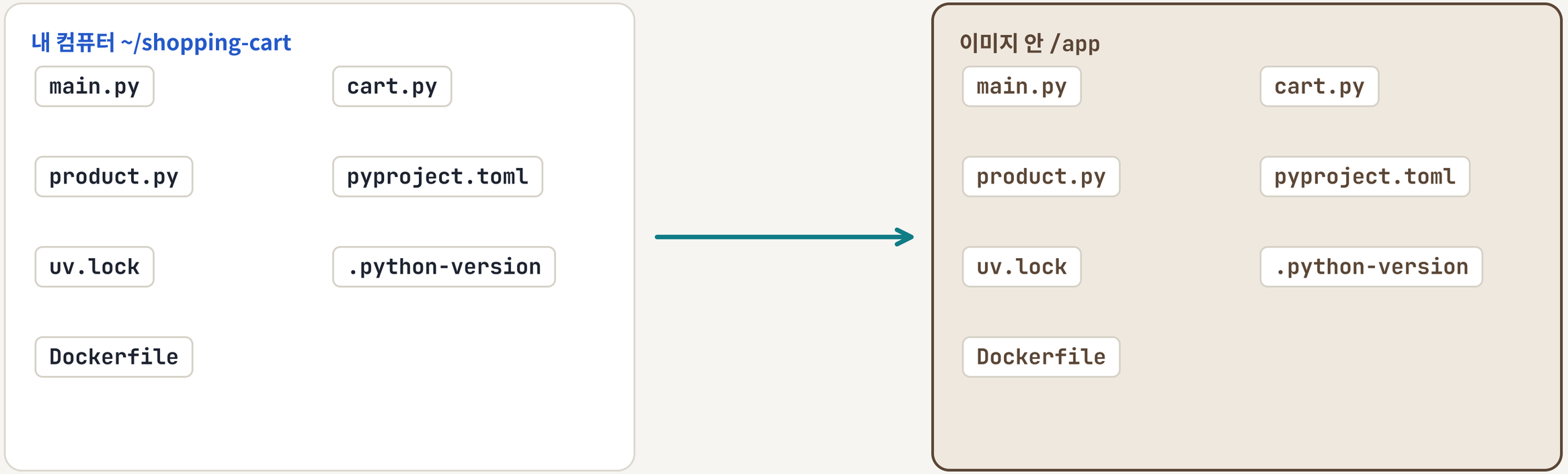
28 프로젝트의 버전과 FROM의 python 숫자를 맞춰요.

3. Dockerfile로 이미지 만들기

WORKDIR과 COPY: 내 파일을 이미지 안 /app에 넣어요

```
WORKDIR /app
COPY . .
```

지금 폴더를 일하는 폴더(/app)로



이제 이미지 안에서 uv sync, uv run main.py를 칠 수 있어요.

3. Dockerfile로 이미지 만들기

RUN은 틀을 만들 때 한 번, CMD는 찍을 때마다 실행돼요

docker build

틀 만들 때 한 번

```
RUN uv sync
```



이미지 cart

- RUN uv sync 결과 (.venv)
- … COPY . .

결과가 틀 안에 구워져요

docker run

붕어빵 찍을 때마다

```
CMD ["uv", "run", "main.py"]
```



컨테이너 cart
uv run main.py
1번째 실행

컨테이너 cart
uv run main.py
2번째 실행

컨테이너 cart
uv run main.py
3번째 실행

CMD는 틀에 적어만 뒀다가 붕어빵을 찍을 때마다 실행해요.

3. Dockerfile로 이미지 만들기

docker build -t cart .로 틀을 구워요

```
docker   build   -t   cart   .
```

- `build` — 틀 굽기
- `cart` — 이미지 이름
- `.` — 재료가 있는 폴더 (지금 폴더)

```
~/shopping-cart $ docker build -t cart .
[+] Building 5.6s (9/9) FINISHED                          docker:default
 => [internal] load build definition from Dockerfile                0.0s
 => [internal] load metadata for docker.io/astral/uv:python3.14-bookworm-  0.9s
 => [internal] load .dockerignore                                   0.0s
 => [1/4] FROM docker.io/astral/uv:python3.14-bookworm-slim@sha256:7cf77f  2.0s
 => [internal] load build context                                   0.0s
 => [2/4] WORKDIR /app                                              0.2s
 => [3/4] COPY . .                                                  0.0s
 => [4/4] RUN uv sync                                               0.7s
 => exporting to image                                              1.6s
 => => naming to docker.io/library/cart:latest                      0.0s
```

- 끝나면 FINISHED
- Dockerfile 1~4줄
  CMD 줄은 적어만 둬요
- 이름이 cart

3. Dockerfile로 이미지 만들기

docker run cart로 내가 구운 틀에서 붕어빵을 찍어요

```
~/shopping-cart $ docker images
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
cart:latest    e89820ea2ada       259MB         67.5MB
nginx:latest   165ddde8751e       243MB         66.8MB
```

```
~/shopping-cart $ docker run cart
텀블러 2개를 담았습니다.
무선 이어폰 1개를 담았습니다.
품절된 상품입니다.
---------------------------------------
텀블러         x 2    =       30,000원
무선 이어폰      x 1    =       89,000원
---------------------------------------
상품 합계                119,000원
배송비                        0원
결제 금액                119,000원
---------------------------------------
결제가 완료되었습니다. 감사합니다!
1: 텀블러           15,000 재고 8개
2: 무선 이어폰        89,000 재고 2개
```

28에서 친 명령과 비교

`uv run main.py`

내 컴퓨터의 파이썬과 uv로

`docker run cart`

이미지 안의 파이썬과 uv로

출력이 한 글자도 다르지 않아요.

코드를 고쳤으면 build부터 다시 해요

main.py

```python
cart = Cart()
cart.add(tumbler, 3)
cart.add(earphone, 1)
```

build할 때 복사한 옛 main.py로 찍었어요

```
~/shopping-cart $ docker run cart
텀블러 2개를 담았습니다.
무선 이어폰 1개를 담았습니다.
품절된 상품입니다.
----------------------------------------
텀블러          x 2    =        30,000원
```

```
~/shopping-cart $ docker build -t cart .
[+] Building 0.8s (9/9) FINISHED                                docker:default
 => [1/4] FROM docker.io/astral/uv:python3.14-bookworm-slim@sha256:7cf77f  0.0s
 => CACHED [2/4] WORKDIR /app                                          0.0s
 => [3/4] COPY . .                                                     0.0s
 => [4/4] RUN uv sync                                                  0.4s
~/shopping-cart $ docker run cart
텀블러 3개를 담았습니다.
```

그대로 써요

바뀐 COPY부터 다시

이제 3개

3. Dockerfile로 이미지 만들기

run할 때마다 컨테이너가 쌓여요. --rm을 붙이면 끝나고 지워져요

```
~/shopping-cart $ docker ps -a
CONTAINER ID   IMAGE          COMMAND             CREATED                  STATUS                              PORTS     NAMES
aea78cc5e078   cart           "uv run main.py"    Less than a second ago   Exited (0) Less than a second ago             loving_dewdney
7407a6f475c8   e89820ea2ada   "uv run main.py"    2 seconds ago            Exited (0) 1 second ago                       charming_aryabhata
81b56e0ac052   e89820ea2ada   "uv run main.py"    3 seconds ago            Exited (0) 2 seconds ago                      distracted_elbakyan
```

Exited 컨테이너 3개

```
~/shopping-cart $ docker run --rm cart
텀블러 3개를 담았습니다.

…
~/shopping-cart $ docker ps -a
CONTAINER ID   IMAGE          COMMAND             CREATED          STATUS                              PORTS     NAMES
aea78cc5e078   cart           "uv run main.py"    1 second ago     Exited (0) Less than a second ago             loving_dewdney
7407a6f475c8   e89820ea2ada   "uv run main.py"    3 seconds ago    Exited (0) 2 seconds ago                      charming_aryabhata
81b56e0ac052   e89820ea2ada   "uv run main.py"    4 seconds ago    Exited (0) 2 seconds ago                      distracted_elbakyan
```

--rm으로 찍은 붕어빵은 목록에 늘지 않아요. 영수증처럼 찍고 끝나는 프로그램에 붙여요.

3. Dockerfile로 이미지 만들기

.dockerignore에 적은 파일은 이미지에 넣지 않아요

.dockerignore

```
1  .venv          내 컴퓨터용 가상환경
2  __pycache__    파이썬이 만든 임시 파일
3  .git           Git 기록
```

이미지 안의 .venv는 RUN uv sync가
리눅스용으로 새로 만든 거예요.

build할 때 보내는 재료 (transferring context)

83.15kB → 300B

없을 때 /app

```
.git
.gitignore
.python-version
.venv
Dockerfile
__pycache__
cart.py
main.py
product.py
pyproject.toml
uv.lock
```

있을 때 /app

```
.dockerignore
.gitignore
.python-version
.venv
Dockerfile
cart.py
main.py
product.py
pyproject.toml
uv.lock
```

4. docker compose 맛보기

긴 docker run을 compose.yaml에 옮겨 적어요

지금까지 친 명령

```
docker run -d --name web -p 8080:80 -v ./site:/usr/share/nginx/html nginx
```

compose.yaml

```yaml
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    volumes:
      - ./site:/usr/share/nginx/html
```

nginx

-p 8080:80

-v ./site:…

services 아래 **web**이 컨테이너 하나예요.
들여쓰기는 2칸씩, 목록은 -로 시작해요.

~/cafe 폴더 맨 위,
site 폴더 옆에 둬요.

한 번 적어 두면 다음부터는 짧은 명령 두 개로 켜고 꺼요.

4. docker compose 맛보기

docker compose up -d로 켜고 down으로 지워요

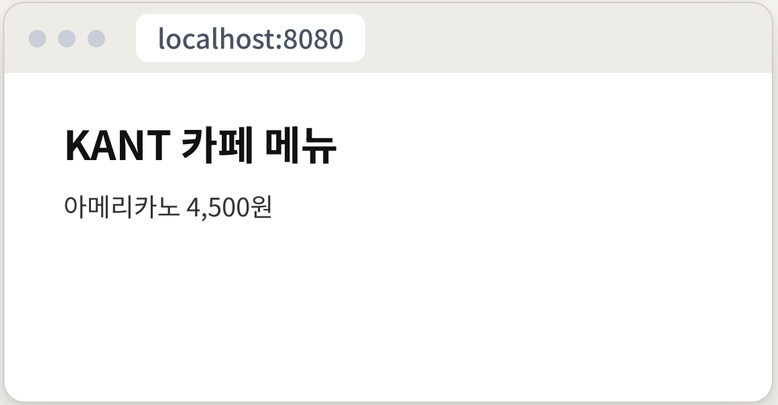
```
~/cafe $ docker compose up -d
[+] up 2/2                                                    -d 그대로, 뒤에서 돌아요
 ✔ Network cafe_default Created                          0.0s
 ✔ Container cafe-web-1 Started                          0.2s

~/cafe $ docker compose ps
NAME          IMAGE     COMMAND                  SERVICE   CREATED         STATUS                   PORTS
cafe-web-1    nginx     "/docker-entrypoint.…"   web       1 second ago    Up Less than a second    0.0.0.0:8080->80/tcp

~/cafe $ docker compose down                                  stop과 rm을 한 번에
[+] down 2/2
 ✔ Container cafe-web-1 Removed                          0.2s
 ✔ Network cafe_default Removed                          0.2s
```

이름: 폴더 이름-서비스 이름-번호

Network 줄은 compose가 만든 연결망이에요. 이 챕터에서는 넘어가요.



5. 자주 나는 오류

build 오류는 마지막 ERROR 줄부터 읽어요

```
~/shopping-cart $ docker build -t cart
ERROR: docker: 'docker buildx build' requires 1 argument
```

끝의 점(.)을 빼먹었어요 **docker build -t cart .**

```
~ $ docker build -t cart .
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
```

Dockerfile이 없는 폴더에서 쳤어요 **cd shopping-cart**

```
~/shopping-cart $ docker build -t cart .
   1 | >>> FROM astral/uv:python3.14-bookwrom-slim
ERROR: failed to build: failed to solve: astral/uv:python3.14-bookwrom-slim: failed to resolve source metadata for docker.io/astral/
uv:python3.14-bookwrom-slim: docker.io/astral/uv:python3.14-bookwrom-slim: not found
```

FROM의 이미지 이름이나 태그에 오타가 있어요 **bookwrom → bookworm**

5. 자주 나는 오류

run과 exec 오류는 원인이 한 줄에 다 적혀 있어요

```
~/shopping-cart $ docker run --rm cart
error: Failed to spawn: `mian.py`
  Caused by: No such file or directory (os error 2)
```

CMD의 파일 이름 오타 (mian.py)

Dockerfile 고치고 다시 build

```
~/cafe $ docker exec -it web bash
Error response from daemon: container b0ed4b95660e… is not running
```

멈춘 컨테이너에는 못 들어가요

docker start web

```
~/cafe $ docker run -d --name web -p 8080:80 nginx
docker: Error response from daemon: … Bind for 0.0.0.0:8080 failed: port is already allocated
```

8080을 다른 컨테이너가 쓰고 있어요

docker compose down 또는 -p 8081:80

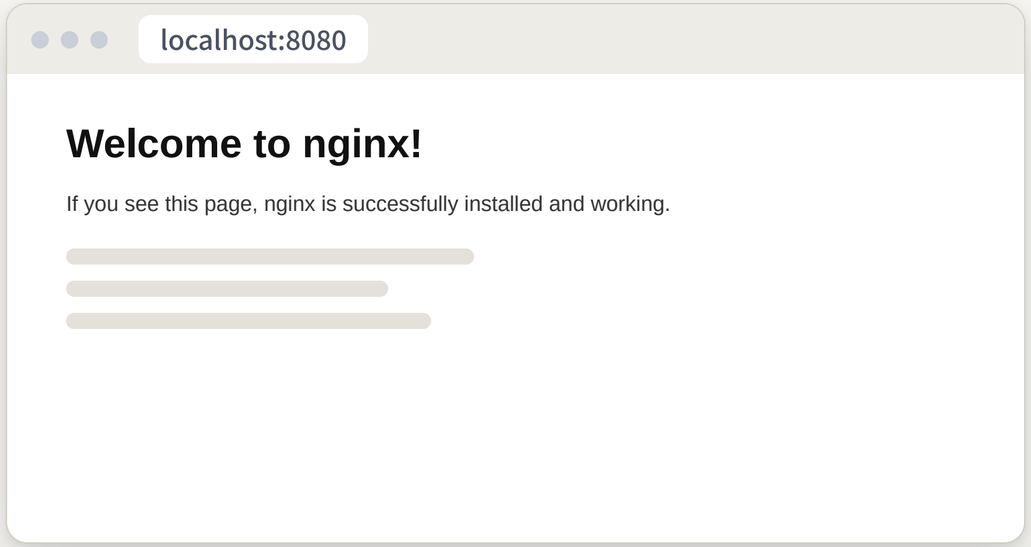
5. 자주 나는 오류

-v에서 ./를 빼면 폴더가 아니라 새 볼륨이 생겨요

```
~/cafe $ docker run -d --name web -p 8080:80 -v site:/usr/share/nginx/html nginx
be746d49b581…

~/cafe $ docker volume ls
DRIVER    VOLUME NAME
local     cafe-html
local     site
```

site라는 볼륨이 생겼어요



메뉴 대신 nginx 환영 페이지가 나와요.

`-v ./site:/usr/share/nginx/html`

./를 붙여 다시 만들고, 잘못 생긴 볼륨은
docker volume rm site로 지워요.

```
~ $ docker compose up -d
no configuration file provided: not found
```

yaml 없는 폴더

6. 실제 상황 예제

예제 1 · 카페 메뉴판을 compose로 띄우고 고쳐요

1. **`cd cafe`**
   compose.yaml이 있는 폴더로
2. **`docker compose up -d`**
   켜기
3. **site/index.html 고치고 저장**
   VS Code에서
4. **브라우저 새로 고침**
   localhost:8080
5. **`docker compose down`**
   끄고 지우기

site/index.html

```
1  <meta charset="utf-8">
2  <h1>KANT 카페 메뉴</h1>
3  <p>아메리카노 4,500원</p>
4  <p>카페라테 5,000원</p>
```



6. 실제 상황 예제

예제 2 · 친구는 clone하고 build, run 두 줄이면 돼요

내가 할 일

Dockerfile, .dockerignore 커밋

git add 파일 뒤 git commit -m

`git push origin main`

02에서 배운 그대로

친구가 할 일

```
~ $ git clone https://github.com/kim/shopping-cart.git
~ $ cd shopping-cart
~/shopping-cart $ docker build -t cart .
…
~/shopping-cart $ docker run --rm cart
텀블러 3개를 담았습니다.
무선 이어폰 1개를 담았습니다.
…
```

Docker만 있으면 돼요

이미지 cart

```
CMD uv run main.py
RUN uv sync
COPY . .
FROM astral/uv…
```

친구 컴퓨터에 없어도 되는 것

- `python` 이미지 안에 있어요
- `uv` 이미지 안에 있어요
- `.venv` build할 때 만들어요

6. 실제 상황 예제

예제 3 · 다 쓴 컨테이너, 이미지, 볼륨을 정리해요

```
~/shopping-cart $ docker ps -a
CONTAINER ID   IMAGE          COMMAND            CREATED          STATUS                      PORTS     NAMES
aea78cc5e078   8900ed2bf435   "uv run main.py"   16 seconds ago   Exited (0) 15 seconds ago             loving_dewdney
7407a6f475c8   e89820ea2ada   "uv run main.py"   18 seconds ago   Exited (0) 17 seconds ago             charming_aryabhata
81b56e0ac052   e89820ea2ada   "uv run main.py"   19 seconds ago   Exited (0) 17 seconds ago             distracted_elbakyan
~/shopping-cart $ docker rm loving_dewdney charming_aryabhata distracted_elbakyan
loving_dewdney
charming_aryabhata
distracted_elbakyan
~/shopping-cart $ docker rmi cart
Untagged: cart:latest
Deleted: sha256:16431c0b53eb814336670b8cd825087b0e9b9d3ec8837074879dc939291c642c
~/shopping-cart $ docker volume rm cafe-html site
cafe-html
site
```

1 컨테이너 (붕어빵)

2 이미지 (틀)

3 볼륨 (보관함)

붕어빵부터 지우고 틀과 보관함을 지워요. 지우면 되돌릴 수 없으니 이름을 한 번 더 봐요.

정리

막히면 이렇게 풀어요

이런 글이 보이면	이렇게 해요
`Error ... container ... is not running` 멈춘 컨테이너에 exec 했어요	`docker start 이름`
`ERROR: ... requires 1 argument` docker build 끝의 점(.)이 없어요	`docker build -t cart .`
`open Dockerfile: no such file or directory` Dockerfile이 없는 폴더예요	`pwd 뒤 cd shopping-cart`
`port is already allocated` 다른 컨테이너가 그 포트를 써요	`docker ps 뒤 stop, down`
코드를 고쳤는데 결과가 그대로예요 이미지는 build할 때의 코드예요	`docker build 다시`
Git Bash에서 경로에 C:/Program Files/Git가 붙어요 Git Bash가 /usr/...를 바꿨어요	`export MSYS_NO_PATHCONV=1`

정리

한 장 요약

안에 들어가기, 남기기

`docker exec -it 이름 bash`
실행 중인 컨테이너 안으로

`-v 볼륨이름:경로`
지워도 남는 보관함

`-v ./폴더:경로`
내 폴더 연결

`docker volume ls / rm`
볼륨 보기, 지우기

이미지 만들기

`FROM WORKDIR COPY RUN CMD`
Dockerfile 다섯 줄

`docker build -t 이름 .`
틀 굽기 (코드 고치면 다시)

`docker run --rm 이름`
찍고 끝나면 지우기

`.dockerignore`
.venv __pycache__ .git

compose

`compose.yaml`
image, ports, volumes

`docker compose up -d`
켜기

`docker compose ps`
이 폴더 것만 보기

`docker compose down`
끄고 지우기

이미지는 틀, 컨테이너는 붕어빵, 볼륨은 붕어빵 밖 보관함이에요.

오늘 만든 cart 이미지를 친구 컴퓨터에서도 돌려 봐요.